agentic-ai-azure-week10-observability

Week 10 - Observability, Evaluation & Governance

Agentic AI on Azure - Enterprise Master Class | Handout (slides + notes)

Notes

Welcome. This deck covers 'agentic-ai-azure-week10-observability' - Week 10 - Observability, Evaluation & Governance. Frame the problem and why it matters, then walk the class through the learning goal, the enterprise use case, the architecture/flow, the key concepts, a live run in VS Code, and the architect's takeaways. The repo runs locally with no Azure, so demo it live.

Slide 2 - Learning goal

- Trace every interaction end-to-end
- Evaluate quality (groundedness, relevance, safety)
- Flag drops and feed a dashboard

Notes

[Learning goal] Walk through these talking points:
- Trace every interaction end-to-end
- Evaluate quality (groundedness, relevance, safety)
- Flag drops and feed a dashboard
Ask the class: which of these ideas is new to you?

Slide 3 - Enterprise use case

- Regulated Advice Quality Monitoring
- Every answer is traced and scored
- Auditors review a quality dashboard

Notes

[Enterprise use case] Walk through these talking points:
- Regulated Advice Quality Monitoring
- Every answer is traced and scored
- Auditors review a quality dashboard
Tip: relate this to a regulated scenario your students know.

Slide 4 - Architecture / flow

- Per-request trace id + span (OpenTelemetry)
- Eval scoring of each answer
- Flag answers below threshold
- Aggregate metrics at /api/v1/metrics

Notes

[Architecture / flow] Walk through these talking points:
- Per-request trace id + span (OpenTelemetry)
- Eval scoring of each answer
- Flag answers below threshold
- Aggregate metrics at /api/v1/metrics
Demo: trace a single request through these steps live.

Slide 5 - Key concepts

- OpenTelemetry -> Application Insights (optional)
- Foundry Evaluations in prod; heuristics here
- Offline eval (CI gates) vs. online sampling

Notes

[Key concepts] Walk through these talking points:
- OpenTelemetry -> Application Insights (optional)
- Foundry Evaluations in prod; heuristics here
- Offline eval (CI gates) vs. online sampling
Open the named file in the repo while you explain each point.

Slide 6 - Run it (VS Code - no Azure needed)

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/advise
- Runs in MOCK mode - no Azure/keys needed

Notes

[Run it (VS Code - no Azure needed)] Walk through these talking points:
- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/advise
- Runs in MOCK mode - no Azure/keys needed
Pause for questions before moving on.

Slide 7 - Architect's takeaways

- Trace/span per agent hop and tool call
- Golden datasets, A/B prompts, drift detection
- Cost & latency per token / per tool

Notes

[Architect's takeaways] Walk through these talking points:
- Trace/span per agent hop and tool call
- Golden datasets, A/B prompts, drift detection
- Cost & latency per token / per tool
Discussion: when would you NOT choose this approach?